

```

# ROS 2 node for the command handle
fleet_name = fleet_config.fleet_name
node = rclpy.node.Node(f'{fleet_name}_command_handle')
adapter = Adapter.make(f'{fleet_name}_fleet_adapter')
assert adapter, (
    'Unable to initialize fleet adapter. '
    'Please ensure RMF Schedule Node is running'
)

# Enable sim time for testing offline
if args.use_sim_time:
    param = Parameter("use_sim_time", Parameter.Type.BOOL, True)
    node.set_parameters([param])
    adapter.node.use_sim_time()

adapter.start()
time.sleep(1.0)

if args.server_uri == '':
    server_uri = None
else:
    server_uri = args.server_uri

fleet_config.server_uri = server_uri
fleet_handle = adapter.add_easy_fleet(fleet_config)

```

b. Configure transformations between RMF and robot

We have defined a helper function to compute the transforms between RMF and the robot's coordinates. In the event your robot operates in the same coordinates as RMF (e.g. in simulation), you won't need this portion of the code.

```

def compute_transforms(level, coords, node=None):
    """Get transforms between RMF and robot coordinates."""
    rmf_coords = coords['rmf']
    robot_coords = coords['robot']
    tf = nugged.estimate(rmf_coords, robot_coords)
    if node:
        mse = nugged.estimate_error(tf, rmf_coords, robot_coords)
        node.get_logger().info(
            f"Transformation error estimate for {level}: {mse}"
        )

    return Transformation(
        tf.get_rotation(),
        tf.get_scale(),
        tf.get_translation()
    )

```

```
# Configure the transforms between robot and RMF frames
for level, coords in config_yaml['reference_coordinates'].items():
    tf = compute_transforms(level, coords, node)
    fleet_config.add_robot_coordinates_transformation(level, tf)
```

Depending on the number of maps (or levels) required for your integration, you will extract the corresponding coordinate transformations for each map and add them to the FleetConfiguration object. The transformation error estimate will be logged by this function if you pass your `rclpy.Node` into it.

Then, in our `main` function, we add the computed transforms to our FleetConfiguration. The EasyFullControl fleet adapter will process these transforms and send out navigation commands in the robot's coordinates accordingly.

```
# Configure the transforms between robot and RMF frames
for level, coords in config_yaml['reference_coordinates'].items():
    tf = compute_transforms(level, coords, node)
    fleet_config.add_robot_coordinates_transformation(level, tf)
```

c. Initialize the robot API and set up RobotAdapter

The `config.yaml` may include any connection credentials we'd need to connect to our robot or robot fleet manager. We parse this to the `RobotAPI` to easily interact between RMF and the robot's API. This is entirely optional; for more secure storage of credentials, do import them into RobotAPI accordingly.

```
# Initialize robot API for this fleet
fleet_mgr_yaml = config_yaml['fleet_manager']
api = RobotAPI(fleet_mgr_yaml)
```

Given a list of known robots from our `config.yaml`, we can initialize a `RobotAdapter` class for each robot that is supposed to be added to the fleet.

```
robots = {}
for robot_name in fleet_config.known_robots:
    robot_config = fleet_config.get_known_robot_configuration(robot_name)
    robots[robot_name] = RobotAdapter(
        robot_name, robot_config, node, api, fleet_handle
    )
```

d. Retrieve robot status and add robot to the fleet

This update loop will allow us to update the `RobotUpdateHandle` with our robots' information asynchronously, such that any error in retrieving the status from one robot won't block the other robots from updating the fleet adapter.

```
update_period = 1.0/config_yaml['rmf_fleet'].get(
    'robot_state_update_frequency', 10.0
)

def update_loop():
    asyncio.set_event_loop(asyncio.new_event_loop())
    while rclpy.ok():
        now = node.get_clock().now()

        # Update all the robots in parallel using a thread pool
        update_jobs = []
        for robot in robots.keys():
            update_jobs.append(update_robot(robot))

        asyncio.get_event_loop().run_until_complete(
            asyncio.wait(update_jobs)
        )

        next_wakeup = now + Duration(nanoseconds=update_period*1e9)
        while node.get_clock().now() < next_wakeup:
            time.sleep(0.001)

update_thread = threading.Thread(target=update_loop, args=())
update_thread.start()
```

The function `update_robot()` is called to ensure that our robots' current map, position and battery state of charge will be updated properly. If the robot is new to the fleet handle, we will add it in via `add_robot()`.

```

@parallel
def update_robot(robot: RobotAdapter):
    data = robot.api.get_data(robot.name)
    if data is None:
        return

    state = rmf_easy.RobotState(
        data.map,
        data.position,
        data.battery_soc
    )

    if robot.update_handle is None:
        robot.update_handle = robot.fleet_handle.add_robot(
            robot.name,
            state,
            robot.configuration,
            robot.make_callbacks()
        )
    return

robot.update(state)

```

e. Inside the RobotAdapter class

The `RobotAdapter` class helps us to keep track of any ongoing process the robot may be carrying out, and perform the correct actions when RMFs sends a corresponding command.

```

class RobotAdapter:
    def __init__(
        self,
        name: str,
        configuration,
        node,
        api: RobotAPI,
        fleet_handle
    ):
        self.name = name
        self.execution = None
        self.update_handle = None
        self.configuration = configuration
        self.node = node
        self.api = api
        self.fleet_handle = fleet_handle

```

There are 3 important callbacks that we need to pass on to the EasyFullControl API:

- navigate

- stop
- execute_action

As described above, each of these callbacks will be triggered by RMF when it needs to command to robot to do something. Hence, we define these callbacks in our `RobotAdapter` :

```
def navigate(self, destination, execution):
    self.execution = execution
    self.node.get_logger().info(
        f'Commanding [{self.name}] to navigate to {destination.position} '
        f'on map [{destination.map}]'
    )

    self.api.navigate(
        self.name,
        destination.position,
        destination.map,
        destination.speed_limit
    )

def stop(self, activity):
    if self.execution is not None:
        if self.execution.identifier.is_same(activity):
            self.execution = None
            self.stop(self.name)

def execute_action(self, category: str, description: dict, execution):
    ''' Trigger a custom action you would like your robot to perform.
    You may wish to use RobotAPI.start_activity to trigger different
    types of actions to your robot.'''
    self.execution = execution
    # ----- #
    # IMPLEMENT YOUR CODE HERE #
    # ----- #
    return
```

Notice that `execute_action(~)` does not have any implemented code in the fleet adapter template. This callback is designed to be flexible and caters to custom performable actions that may not be available under the tasks offered in RMF. You can learn how to design and compose your own actions and execute them from the fleet adapter in the [PerformAction tutorial](#) section.

```
def make_callbacks(self):  
    return rmf_easy.RobotCallbacks(  
        lambda destination, execution: self.navigate(  
            destination, execution  
        ),  
        lambda activity: self.stop(activity),  
        lambda category, description, execution: self.execute_action(  
            category, description, execution  
        )  
    )
```

Finally, we add all of our callbacks to our fleet adapter using the `RobotCallbacks()` API.

Found a bug? [Edit this page on GitHub](#).

PerformAction Tutorial (Python)

This tutorial is an extension of the Fleet Adapter Tutorial and will guide you to write custom actions in your fleet adapter. While RMF offers a few standard tasks, we understand that different robots may be equipped and programmed to perform different types of actions, such as cleaning, object-picking, teleoperation, and so on. By supporting custom tasks, users can trigger a custom action specified in the fleet adapter's `config.yaml` beforehand, and RMF would relinquish control of the robot until it is signalled that the robot has completed the custom action. You may explore the [Supporting a new Task in RMF](#) section to read more about supporting custom tasks and how you can create your own task JSON to be sent to RMF.

In this tutorial, we will refer to a simplified version of the `rmf_demos_fleet_adapter` to implement a `Clean` PerformAction capability in our fleet adapter.

1. Define the PerformAction in the fleet config.yaml

We will need to define the name of the action in the fleet configuration, so that RMF recognizes this action as performable when a task is submitted and is able to dispatch it to a fleet that can fulfil it. In our `config.yaml` under the `rmf_fleet` section, we can provide a list of performable actions for our fleet. For example, let's define `clean` as an action supported by this fleet:

```
rmf_fleet:  
  actions: ["clean"]
```

2. Apply action execution logic inside our fleet adapter

After RMF receives a task consisting of this action and dispatches it to the right fleet, the fleet adapter's `execute_action(~)` callback will be triggered. The `category` parsed to this callback corresponds to the action name that we have previously defined, and the `description` consists of any details about the action that we might be interested in.

Assume that this is the task JSON submitted to RMF:

```
{
  "type": "dispatch_task_request",
  "request": {
    "unix_millis_earliest_start_time": start_time,
    "category": "clean",
    "description": {
      "zone": "clean_lobby"
    }
  }
}
```

In our example, the `category` provided would be `clean`, and the `description` would contain which cleaning zone this task is directing our robot to, which is `clean_lobby`. Hence, we will need to implement the logic in our `execute_action(~)`:

```
def execute_action(self, category: str, description: dict, execution):
    self.execution = execution

    if category == 'clean':
        self.perform_clean(description['zone'])

def perform_clean(self, zone):
    if self.api.start_activity(self.name, 'clean', zone):
        self.node.get_logger().info(
            f'Commanding [{self.name}] to clean zone [{zone}]'
        )
    else:
        self.node.get_logger().error(
            f'Fleet manager for [{self.name}] does not know how to '
            f'clean zone [{zone}]. We will terminate the activity.'
        )
    self.execution.finished()
    self.execution = None
```

Since our fleet may be capable of performing multiple custom actions, we will need to conduct a check to ensure that the `category` received matches the robot API that we are targeting. Upon receiving a `clean` action, we can trigger the robot's API accordingly.

3. Implement the robot API for the custom action

This is where the `start_activity(~)` method inside `RobotClientAPI.py` comes into play. We would require it to implement the API call to the robot to start the cleaning activity. As an example, if the robot API uses REST to make calls to the robot, the implemented method may look like this:


```

def start_activity(
    self,
    robot_name: str,
    activity: str,
    label: str
):
    ''' Request the robot to begin a process. This is specific to the robot
        and the use case. For example, load/unload a cart for Deliverybot
        or begin cleaning a zone for a cleaning robot.'''
    url = (
        self.prefix +
        f"/open-rmf/rmf_demos_fm/start_activity?robot_name={robot_name}"
    )
    # data fields: task, map_name, destination{}, data{}
    data = {'activity': activity, 'label': label}
    try:
        response = requests.post(url, timeout=self.timeout, json=data)
        response.raise_for_status()
        if self.debug:
            print(f'Response: {response.json()}')

        if response.json()['success']:
            return True

        # If we get a response with success=False, then
        return False
    except HTTPError as http_err:
        print(f'HTTP error for {robot_name} in start_activity: {http_err}')
    except Exception as err:
        print(f'Other error {robot_name} in start_activity: {err}')
    return False

```

4. Complete the action

Since we stored a `self.execution` object in our `RobotAdapter`, we will be notified when any execution (navigation, stop, or action) is completed as the update loop continually calls `is_command_completed` to check on its status.

```

def update(self, state):
    activity_identifier = None
    if self.execution:
        if self.api.is_command_completed():
            self.execution.finished()
            self.execution = None
        else:
            activity_identifier = self.execution.identifier

```

If your implementation requires a separate callback to mark the execution as finished, you can create a new function to conduct this check and call `self.execution.finished()` when the action is completed.

Found a bug? [Edit this page on GitHub.](#)